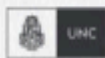


CERTIFICACIÓN UNIVERSITARIA



Manual para el
alumno

Terraform II



**WE WANT
SKILLS!**

mundosE

Manual para el alumno: Terraform II.....	2
Objetivo del encuentro:.....	2
1. Introducción al HashiCorp Configuration Language (HCL).....	3
2. Proveedores en Terraform.....	4
3. Archivo de estado (terraform.tfstate) y riesgos del estado local.....	5
4. Backends remotos: S3, Azure Blob Storage y Terraform Cloud.....	6
Bibliografía.....	7

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En este encontrarás un resumen de cada tema tratado en el encuentro, así como también, recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el alumno: Terraform II

Objetivo del encuentro:

Profundizar en el uso de Terraform comprendiendo la sintaxis HCL, la organización de archivos, la importancia de los proveedores, el funcionamiento del archivo de estado y las estrategias de gestión remota para garantizar proyectos reproducibles y seguros.

Contenido a desarrollar:

- ✓ **Introducción al HashiCorp Configuration Language (HCL):**
Conceptos de bloques, atributos y sintaxis declarativa.
- ✓ **Proveedores en Terraform:**
Rol de los *providers* en la conexión con APIs externas.
- ✓ **Ejemplos de proveedores:**
AWS, Azure, Google Cloud Platform y Kubernetes.
- ✓ **Archivo de estado (**terraform.tfstate**):**
Importancia en la trazabilidad y control de la infraestructura.
- ✓ **Riesgos del estado local:**
Pérdida de información, problemas de concurrencia y exposición de datos sensibles.
- ✓ **Backends remotos en Terraform:**
Uso de S3, Azure Blob Storage y Terraform Cloud para gestión colaborativa del estado.

1. Introducción al HashiCorp Configuration Language (HCL)

Terraform utiliza el **HashiCorp Configuration Language (HCL)** como su lenguaje de definición de infraestructura. La elección de HashiCorp de crear un lenguaje propio responde a la necesidad de un formato que sea, a la vez, **fácil de leer por humanos** y **estrictamente estructurado para las máquinas**.

En su base, HCL se compone de **bloques**. Cada bloque representa un recurso, un proveedor, una variable, un output u otro componente relevante. Dentro de cada bloque se definen **atributos** y **argumentos** que describen el estado deseado de ese elemento.

Ejemplo sencillo:

```
resource "aws_instance" "web" {  
  
    ami           = "ami-12345678"  
  
    instance_type = "t2.micro"  
  
}
```

Aquí:

- **resource** define el tipo de bloque.
- **"aws_instance"** indica el tipo de recurso a crear (una máquina virtual en AWS).
- **"web"** es el nombre lógico que usaremos como referencia.
- Dentro del bloque se especifican atributos como la AMI (imagen del sistema operativo) y el tipo de instancia.

HCL no es un lenguaje de programación general: no tiene bucles infinitos ni estructuras de control complejas. Pero sí ofrece expresiones básicas como interpolación de variables (**var.nombre**), operadores condicionales (**count**, **for_each**) y funciones integradas. Esto permite generar configuraciones dinámicas sin perder la simplicidad declarativa.

Además de recursos, HCL permite declarar:

- **Variables (`variable {}`):** para parametrizar configuraciones.
- **Outputs (`output {}`):** para exponer valores útiles, como la IP de una máquina creada.
- **Data sources (`data {}`):** para consultar información ya existente en un proveedor (por ejemplo, obtener la última AMI disponible en AWS).

Desde el punto de vista práctico, HCL es más legible que JSON o YAML. Esto facilita la colaboración entre equipos diversos, incluso aquellos que no tienen experiencia en programación. Para los estudiantes, el aprendizaje de HCL es directo: con unos pocos ejemplos se puede comenzar a describir infraestructura real.

Recursos para profundizar:

- HashiCorp. (s. f.). *Configuration Language*.
<https://developer.hashicorp.com/terraform/language>

2. Proveedores en Terraform

El concepto de **proveedor (provider)** es central en Terraform. Un proveedor es un plugin que permite a Terraform comunicarse con la API de un servicio externo. Sin un proveedor, Terraform no sabe con qué infraestructura trabajar.

Ejemplo de definición:

```
provider "aws" {
  region = "us-east-1"
}
```

En este caso, se indica que se usará AWS como proveedor y que los recursos se crearán en la región US-East-1.

Los proveedores se descargan automáticamente cuando se ejecuta **terraform init**. Cada proveedor define qué **recursos** y **data sources** están disponibles. Por ejemplo:

- El proveedor **AWS** permite crear instancias EC2, buckets S3, redes VPC, etc.
- El proveedor **AzureRM** habilita bases de datos, máquinas virtuales y servicios de red en Azure.
- El proveedor **Google (GCP)** permite gestionar recursos en Google Cloud

Platform.

- El proveedor **Kubernetes** se utiliza para gestionar objetos como pods, deployments o servicios dentro de un clúster.

Los proveedores son lo que hacen que Terraform sea **multi-cloud** y evite el vendor lock-in. Una misma organización puede tener recursos distribuidos en varias nubes y gestionarlos de forma unificada con Terraform.

En el aula, este punto es clave: comprender que los proveedores son las “puertas de entrada” a cada ecosistema. Sin ellos, no se puede crear nada.

Recursos para profundizar:

- HashiCorp. (s. f.). *Terraform Providers*.
<https://developer.hashicorp.com/terraform/providers>

3. Archivo de estado (**terraform.tfstate**) y riesgos del estado local

Terraform mantiene un archivo llamado **terraform.tfstate**, que refleja el estado actual de la infraestructura desplegada. Este archivo contiene un mapa detallado de todos los recursos creados, con sus atributos y metadatos.

¿Por qué es importante? Porque Terraform compara el estado actual (archivo de estado) con el estado deseado (archivos **.tf**). Esa comparación le permite determinar qué cambios son necesarios: crear, modificar o eliminar recursos.

Ejemplo: si en los archivos **.tf** hay definida una máquina virtual con cierto nombre, y en el archivo de estado aparece otra diferente, Terraform detectará la diferencia y propondrá actualizar la infraestructura para alinearla.

El archivo de estado es fundamental, pero también trae riesgos:

- **Seguridad:** contiene información sensible (IDs de recursos, direcciones IP, contraseñas codificadas).
- **Concurrencia:** si dos usuarios aplican cambios al mismo tiempo, pueden sobrescribir el estado y generar inconsistencias.
- **Pérdida de datos:** si el archivo se elimina accidentalmente, se pierde la referencia entre la configuración y los recursos existentes.

Por estas razones, no se recomienda mantener el estado en local cuando se trabaja en equipo.

Recursos para profundizar:

- HashiCorp. (s. f.). *Terraform State*.
<https://developer.hashicorp.com/terraform/language/state>

4. Backends remotos: S3, Azure Blob Storage y Terraform Cloud

La solución a los problemas del estado local es utilizar **backends remotos**. Un backend remoto permite almacenar el archivo de estado en un servicio externo, asegurando disponibilidad, seguridad y control de concurrencia.

Los backends más comunes son:

- **AWS S3 con DynamoDB:** el archivo de estado se guarda en un bucket S3, mientras que DynamoDB se usa para el bloqueo (evita que dos usuarios modifiquen al mismo tiempo).
- **Azure Blob Storage:** equivalente en la nube de Microsoft, permite almacenar el estado y gestionar bloqueos con Azure Storage.
- **Terraform Cloud:** servicio de HashiCorp que centraliza el estado, ofrece ejecución remota, integración con Git y políticas de seguridad.

El docente debe remarcar que el uso de backends no es opcional en equipos grandes: es un requisito para garantizar consistencia. Trabajar con estado local puede ser aceptable para un estudiante o un entorno de prueba, pero en producción se necesitan mecanismos robustos de gestión de estado.

Para los alumnos, entender los backends remotos significa comprender cómo escalar Terraform a nivel empresarial. Es la diferencia entre un uso individual y un uso corporativo.

Recursos para profundizar:

- HashiCorp. (s. f.). *Backends*.
<https://developer.hashicorp.com/terraform/language/settings/backends>

Bibliografía

- HashiCorp. (s. f.). *Terraform Documentation*. Recuperado de <https://developer.hashicorp.com/terraform>
- HashiCorp. (s. f.). *Terraform Providers*. Recuperado de <https://developer.hashicorp.com/terraform/providers>
- HashiCorp. (s. f.). *Terraform State*. Recuperado de <https://developer.hashicorp.com/terraform/language/state>
- HashiCorp. (s. f.). *Terraform Backends*. Recuperado de <https://developer.hashicorp.com/terraform/language/settings/backends>